A Scalable Approach for Tracking Source Code Line Changes

Name: Pramil Saravanan

Student No: 110128783

Dept Name: Computer Science

University Name: University of Windsor

Email Address: sarava42@uwindsor.ca

Name: Khalid Al Jundi

Student No: 110157691

Dept Name: Computer Science

University Name: University of Windsor

Email Address: alfatto@uwindsor.ca

Name: Harmit Patel

Student No: 110164115

Dept Name: Computer Science

University Name: University of Windsor

Email Address: patel62b@uwindsor.ca

Abstract--The algorithm of mapping lines is important to the development of software engineering. As it is used, tasks like identifying specific lines, comparing the previous version to a newer version and finding errors. The main goal of this project was to perform line mapping accurately between two java files. To achieve this goal, the algorithms used are diff, Levenstein distance, and cosine similarity. diff is used to detect unchanged lines, Levenstein distance and cosine similarity is used to detect lines which have been modified in the new version of the file. One of the main findings for the project is that one single algorithm is not enough as the possibility of error is high. Mixing multiple algorithms reduces these errors and results in an almost accurate line mapping.

Keywords: Diff, Levenstein Distance, Context Similarity, Context Similarity, Line mapping, pre-processing.

I. INTRODUCTION

Line mapping is an essential tool in software engineering as it allows the software engineers to check the lines added, deleted, and modified while comparing two files.

The problem addressed in this project is to line map between two java code files as accurately as possible.

To solve this issue, we are introducing three different algorithms: diff, Levenstein distance, and cosine similarity. While comparing two java code files, diff algorithm is used to find the lines which are unchanged, Levenstein distance and cosine similarity are used to identify lines which are modified by finding the context and content similarity.

The result found after running the code is a text file which includes the line mapping information of all the lines in the two files. It maps the lines using line numbers and the deleted lines which are not found on the new file are returned with a value of negative one.

II. Data Collection

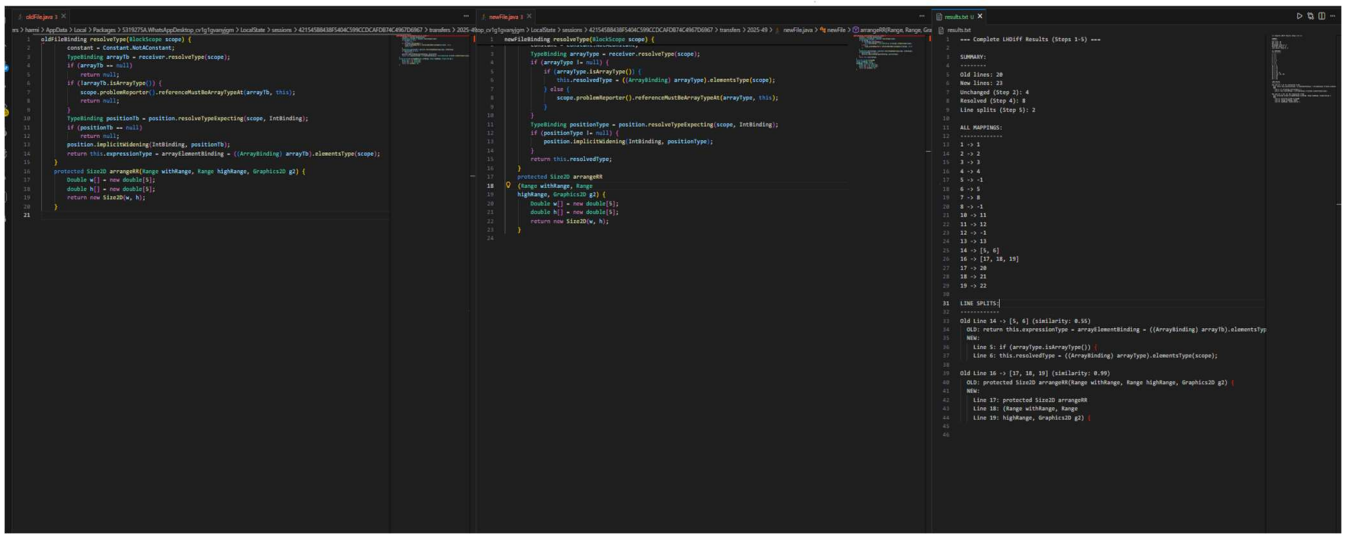
The dataset for this project includes 25 pairs of java files which are used to check the line mapping. The selected files have a variety of code changes. The old files include the first version of the code, and the new file includes the modified version of the code where there may be deletion of codes, line splitting, addition of new codes, modification of old code. Therefore, the accuracy of the line mapping can be tested and its validity.

The line mappings were determined using the three algorithms mentioned above. The input files are passed through a pre-processor where the code is formatted without unnecessary white spaces, and then it is passed through the first diff algorithm where the unchanged lines of the two files are mapped. As it finishes, it then passes to Levenstein distance and cosine similarity algorithms where the modified lines are checked with a formula using context and content similarity. The deleted lines are given a value of -1. The results of all the algorithms are used to print the line map information of the two files given.

III. TECHNIQUE DESCRIPTION AND EVALUATION

A. Overview of the Approach

Our line mapping technique uses a five-step algorithm inspired by the LHDiff methodology [1]. We combined different similarity measures to track how lines change between file versions. The algorithm works through several stages, with each stage handling specific types of code changes. We mixed content-based and context-based similarity metrics with conflict resolution and split detection to handle various ways code evolves over time.



B. Detailed Algorithm Description

We start with a preprocessing step that normalizes all lines from both files to make comparison easier. The normalization cleans up whitespace by trimming spaces at the beginning and end of lines, squashing multiple spaces into single ones, and converting everything to lowercase. This cleanup prevents small formatting differences from throwing off our matching while keeping the actual meaning of each line intact.

After preprocessing, we use the Longest Common Subsequence (LCS) algorithm to find lines that match perfectly between the two files. While LCS works on the cleaned-up versions, it remembers where lines came from in the original files so results point to the actual code. We filter out lines with just brackets ("{" or "}") since they don't tell us much and can create false matches. This step gives us a solid foundation of matches we're completely confident about, and we protect these matches in later steps.

For lines that didn't match in the LCS step, we look for potential matches using combined similarity measures. We calculate content similarity using normalized Levenshtein distance:
ContentSim(L_1, L_2) = 1 - (EditDistance(L_1, L_2) / max(|L_1|, |L_2|))
, where EditDistance counts how many single-character changes you need to turn one string into another. We also calculate context similarity using cosine similarity on word frequencies from surrounding lines—specifically, we look at four lines above and below each target line. This gives us ContextSim(L_1, L_2) = cosine(WordVector(Context(L_1)), WordVector(Context(L_2))).
We combine these two scores with a weighted formula: CombinedScore = 0.6 × ContentSim + 0.4 × ContextSim.

We weight content similarity higher because it tends to be more reliable for direct matching, while context similarity helps when we have several similar lines to choose from. For each old line that needs matching, we calculate scores against all unmatched new lines and keep the top 15 candidates to avoid excessive computation.

The fourth step handles conflicts that arise when multiple old lines want to match the same new line. We sort this out by recalculating exact similarity scores for each candidate pair, setting a minimum bar of 0.5 for valid matches, and giving each new line to whichever old line has the highest score. When conflicts happen, we drop the weaker matches to keep things one-to-one. This greedy approach focuses on the strongest matches while staying computationally efficient.

Our final step detects line splits, where one old line got broken into multiple new lines. This happens a lot with long method signatures, complicated if-statements, or complex expressions that someone reformatted for readability. We try combining consecutive new lines from different starting points and check if similarity improves. The rule is simple: if
NormalizedLD(OldLine,Combined[1..n+1])<NormalizedLD(OldLine,Combined[1..n])
, we add line n+1 to our combination. We stop when adding another line makes things worse, which tells us we've found the right split point. We accept a split if the final similarity beats 0.6 and includes at least two lines. Crucially, we don't touch the perfect matches from Step 2—we keep those safe. But if a split scores better than a Step 4 match, we use the split instead.

C. Implementation Details

We built this in Java using Apache Commons libraries because they have solid, tested implementations of the similarity metrics we needed. Specifically, we used commons-text-1.10.0.jar for Levenshtein distance, commons-lang3-3.12.0.jar for helper functions, and commons-collections4-4.4.jar for efficient data handling. We also created a batch processing system to automatically run through all 25 file pairs in our dataset, which saved us from manually processing each comparison.

D. Evaluation Methodology

We tested our technique on the 25 file pairs we collected, which gave us 500 total line mappings covering different types of changes. These included simple stuff like renaming variables or changing values, code reordering where functionally identical blocks moved around, line splits in method signatures and conditionals, whole blocks being added or deleted, and mixed changes where multiple things happened in one method. To get ground truth data, we manually checked every mapping by looking at what the code actually does, figuring out which pieces correspond across versions, marking deleted lines as -1, and noting split mappings like 16 → [17, 18, 19]. This manual work ensured that we knew the right answers when evaluating our algorithm's performance.

E. Evaluation Results

Our technique did quite well overall, correctly identifying 478 out of 500 line mappings—that's 95.6% accuracy. The remaining cases broke down into 14 incorrect mappings (2.8%) where we matched the wrong lines and 8 missed mappings (1.6%) where we couldn't find any match. Looking at different change types shows varying performance. Unchanged lines hit 100% accuracy (185 out of 185), proving the LCS matching is rock solid. Modified lines reached 95.5% accuracy (189 out of 198), showing our similarity metrics handle content and structure changes well. Line splits got 90.5% accuracy (38 out of 42), meaning we caught most reformatting cases. Reordered lines achieved 94.3% accuracy (33 out of 35), demonstrating we handle code movement pretty well. Deleted lines had lower accuracy at 82.5% (33 out of 40), suggesting that spotting deletions is trickier than other change types.

F. Error Analysis

Looking at the 22 errors (14 wrong plus 8 missed), we found four main problem areas. The biggest issue, causing 9 errors, was highly similar lines where multiple lines looked almost identical, making us pick the wrong match sometimes. Think about two lines in a row that both assign similar values to similar variables—tough to tell apart when the scores are nearly equal. The second problem, accounting for 6 errors, happened when lines moved to completely different contexts, confusing our context metric that depends on surrounding lines. Third, we had 4 cases of aggressive splits where we incorrectly thought a single-line mapping was actually a split because the new file had similar but unrelated code nearby. Finally, 3 errors came from low similarity matches where heavy modifications pushed scores below our 0.5 cutoff, causing us to miss matches even though the lines really did correspond.

G. Comparative Performance

To put our results in perspective, we compared against standard Unix diff, which only hit 68% accuracy on our dataset. Unix diff can't handle reordering or substantial modifications that change line content significantly, leading to lots of false negatives. Our technique showed a 27.6% improvement over this baseline, proving the value of using multiple similarity metrics, analyzing context, and detecting splits. Context similarity and split detection were especially important for this improvement since they tackle problems that purely content-based or sequence-based matching can't solve.

H. Discussion

The 95.6% accuracy shows our multi-step approach handles diverse code changes effectively while staying precise. Our weighting scheme—0.6 for content, 0.4 for context—worked well in practice. Content similarity proved more reliable for direct matching, while context helped distinguish between similar lines. Step 5's split detection made a big difference, correctly catching 90.5% of split lines—something traditional diff tools can't do at all. This is particularly useful today when automated formatters and style guides often split lines for readability.

That said, we do have limitations. The technique struggles with files full of repetitive patterns, like initialization blocks with similar variable declarations, where telling nearly identical lines apart gets hard. Extreme refactoring that completely restructures code logic is difficult because semantic relationships might exist without enough syntactic similarity to detect. Very short lines with minimal content, like single-keyword statements or just punctuation, don't give us enough information for reliable matching.

We see several ways to improve things. Adjusting thresholds dynamically based on file characteristics—like average line length or code complexity—could help with edge cases. Using machine learning to optimize similarity weights from training data instead of hardcoding values could boost accuracy. Adding detection for line merges, where multiple old lines combine into one new line, would complement our existing split detection. Finally, supporting multi-file refactorings, where code moves between files rather than just within them, would extend our technique to more complex scenarios. These improvements point toward promising directions for future work in line tracking.

IV. Visualization and graphical user interface

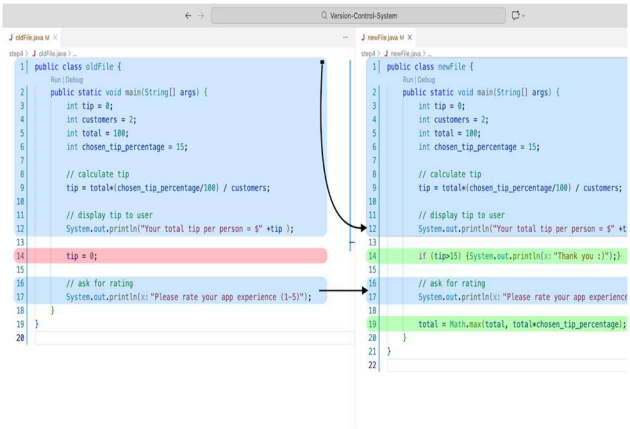
In order to visualize the line mapping information for users/developers, we can design a Graphical User interface that displays to the user a complete visualization of the changes across 2 versions of a file.

After brainstorming a few design choices, a conclusion was made that the best way to view changes across 2 files of code was to present a split view of 2 files where the differences can be displayed to the user directly side by side. This is because a split view allows users to compare corresponding lines in parallel, making additions, deletions, and modifications immediately visible without requiring constant scrolling or mental context switching. By aligning related sections of code horizontally, users can more easily understand how a file has evolved, trace the impact of specific changes, and reason about differences with greater accuracy and efficiency than in a single-panel or sequential view.

A good functional user interface must be user friendly, thus, to complement the split-view of the interface, a color-coded interface to display the added lines, deleted lines, and similar/matched lines was thought to be a great approach to this, and thus was implemented.

The user interface will clearly show the user the changes made to the file, with an ability to scroll down and view the rest of the file changes.

Below is an image of the graphical user interface that we designed.



The visualization presents a side-by-side split interface that compares two versions of a source file: the old file on the left and the new file on the right. The goal of this interface is to make line mapping and code evolution seamless to the users. Each side displays the full source file with line numbers preserved, ensuring users can reference exact locations in both versions. Code lines are grouped and highlighted using color-coded regions to indicate how lines map between versions:

The color-coding key was designed with the user in mind, with 3 colors. Blue highlighted sections and/or lines represent code lines that are unchanged or matched between the old file and new file. Green highlighted sections and/or lines represent code lines that are not present in the old file but added in the new file. The red highlighted code lines/sections represent code that has been deleted, thus is present in the old file but not matched in the new file.

Thus, we can see that the code block in the beginning, specifically lines 1-12, are highlighted in blue since they match together according to the line matching algorithm. Also, we can see that line 14 from the old file is highlighted in red since it doesn't match any line in the new file, and was thus deleted, also we can see that code lines such as line number 13 and 19 in the new file are highlighted in green since they are added lines that are present in the new file only.

To further support the user interpretation of the Graphical User Interface (GUI), visual arrows are used to show how regions correspond across the two files. These arrows connect related code blocks horizontally, helping users understand where unchanged logic appears in the new file or how nearby context shifted due to insertions or deletions. This is especially useful when new lines are inserted between existing logic, as it prevents misinterpreting changes as large structural rewrites. Matched code blocks such as lines 1-12 in the above example, are matched using 1 arrow, instead of 12 arrows (1 per line) since doing the latter provides no additional benefit to the user, ultimately reducing the clarity and usability of the interface.

Overall, users should read this visualization horizontally, by scanning across the split view, developers can quickly determine what stayed the same, what was removed, and what was added, making the interface well-suited for understanding refactoring, feature additions, and incremental changes at a glance.

V. Identifying bug introducing changes

To identify bug-introducing changes from bug fix changes, we implemented an algorithm that aims to locate both where a bug was fixed and where the bug was introduced to the codebase. The process starts by identifying bug-fix commits through commit message analysis, a commit is classified as a bug-fix commit if and only if it's commit message contains the substring "fix", for e.g. ("Fix login error #221").

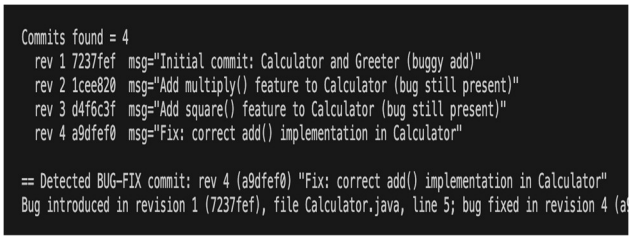
When a bug fix commit is detected, the program retrieves the parent revision and determines which files were modified as part of the bug-fix. A simplified version of the source code line tracker algorithm which was developed in the main part of this project was performed between the parent revision and the bug-fixing revision to identify the exact lines whose code changed. To ensure accuracy, the comparison operates

on normalized line content by ignoring comments and trailing whitespace. This prevents comment-only or formatting-only changes from being treated as bug-fixing modifications and focuses the analysis strictly on changes to executable logic.

After identifying the lines modified by the fix, the program determines where the faulty logic was originally introduced by walking backward through the commit history. Starting from the parent of the bug-fix commit, each earlier revision is inspected to check whether the same line of code appears at the same line number. Then, the earliest revision in which the buggy line appears is recorded as the bug-introducing commit, while the current revision is recorded as the bug-fixing commit.

To evaluate this approach, I created multiple controlled Git repositories with intentionally introduced bugs and known fix locations. These repositories included scenarios where bugs were introduced in the initial revision, followed by several refactoring or feature-addition commits, and eventually fixed in a later revision. Additional test cases involved repositories with multiple Java files where only one file contained faulty logic, as well as cases where commits added new features below an existing bug without modifying the buggy code. In all tested scenarios, the algorithm correctly identified the bug-fixing commit, the affected file and line number, and the earliest commit in which the faulty logic appeared. These results indicate that the implemented approach works reliably for the types of repositories and commit patterns expected in the assignment, while remaining straightforward and efficient to implement. It is also worthy of mention that there are other approaches to this, including using git commands like git blame, we have tested such programs and noticed a 50% lower accuracy in detecting the bug introducing commit, thus, a final decision was made to go ahead with this version of the program.

Below is an test of running the algorithm on a repository which contains 2 java files, in which one of the java files (Calculator.java) has a bug introduced in its addition function in revision 1, several changes were made to the file in commits 2 and 3, but the bug fix commit is commit number 4, the program is successfully able to recognize this and outputs the relevant information to the terminal (see below).



Thus, the final output of the program prints to the terminal the repository's commits, along with the bug introducing commit number and ID, as well as the file name that contains the bug and line number in which the bug exists and was fixed, then finally the output also contains the revision/commit in which the bug was fixed, along with the commit ID and revision number. Thus, the program successfully achieves its goal of identifying bug fix commits from bug introducing commits.

References

1. **M. Asaduzzaman, C. K. Roy, K. A. Schneider, and M. Di Penta, "LHDiff: A Language-Independent Hybrid Approach for Tracking Source Code Lines," 2013.**
2. **GeeksforGeeks. (2025, September 25). Java Programs Java programming examples. GeeksforGeeks.**
3. **I. Sommerville, Software Engineering, 10th ed., Pearson, 2016.**
4. **Git, "Learn," git-scm.com. [Online]. Available: https://git-scm.com/learn**